EduFin SQL Business Challenge

Phase 3 — Wednesday Submission

Student: **Siddhartha Shakya** | shakyasiddhu02@gmail.com

Submitted: 5/1/2026, 10:04:07 AM

STEP 3A — The Scale Shift (Comparison Analysis)

```
SELECT
    dc.city_name AS `City`,
    COUNT(*) AS `Total Loans`,
    ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END), 2) AS `Defaulted Loans`,
    CONCAT(ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2), '%') AS
`Default Rates (%)`
FROM loans_small l
INNER JOIN customers_small c ON l.customer_id = c.customer_id
INNER JOIN dim_city_small dc ON c.city_id = dc.city_id
GROUP BY dc.city_name
HAVING COUNT(*) >= 100
ORDER BY SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*) DESC
LIMIT 10;

SELECT
    dc.city_name AS `City`,
    COUNT(*) AS `Total Loans`,
    ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END), 2) AS `Defaulted Loans`,
    CONCAT(ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2), '%') AS
`Default Rates (%)`
FROM loans_national l
INNER JOIN customers_national c ON l.customer_id = c.customer_id
INNER JOIN dim_city_national dc ON c.city_id = dc.city_id
GROUP BY dc.city_name
HAVING COUNT(*) >= 100
ORDER BY SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*) DESC
LIMIT 10;
```

STEP 3B — CIBIL Score Distribution Among Defaulters

```sql
WITH cibil_data AS (
    SELECT
        c.customer_id,
        c.cibil_score,
        c.full_name,
        l.loan_status,
        CASE
            WHEN c.cibil_score >= 750 AND c.cibil_score <= 900 THEN 'Excellent (750-900)'
            WHEN c.cibil_score >= 650 THEN 'Good (650-749)'
            WHEN c.cibil_score >= 550 THEN 'Fair (550-649)'
            ELSE 'Poor (<550)'
        END AS `CIBIL Brackets`
    FROM customers c
    INNER JOIN loans l
        ON c.customer_id = l.customer_id
)
SELECT
    `CIBIL Brackets`,
    COUNT(*) AS `Total Loans`,
    SUM(CASE WHEN loan_status = 'Defaulted' THEN 1 ELSE 0 END) AS `Defaulted Loans`,
    CONCAT(ROUND(SUM(CASE WHEN loan_status = 'Defaulted' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2), '%') AS
`Default Rate (%)`
FROM cibil_data
GROUP BY `CIBIL Brackets`
ORDER BY SUM(CASE WHEN loan_status = 'Defaulted' THEN 1 ELSE 0 END) DESC;
```

CIBIL Brackets	Total Loans	Defaulted Loans	Default Rate (%)
Good (650-749)	2364	250	10.58%
Fair (550-649)	1218	217	17.82%
Excellent (750-900)	1278	84	6.57%
Poor (<550)	140	39	27.86%

STEP 3C — Open Metric Design: "Risk Efficiency"

```sql
WITH cibil_data AS (
    SELECT
        l.loan_status,
        l.loan_amount,
        CASE
```

```
            WHEN c.cibil_score >= 750 AND c.cibil_score <= 900 THEN 'Excellent (750-900)'
            WHEN c.cibil_score >= 650 THEN 'Good (650-749)'
            WHEN c.cibil_score >= 550 THEN 'Fair (550-649)'
            ELSE 'Poor (<550)'
        END AS `CIBIL Brackets`
    FROM customers c
    INNER JOIN loans l
        ON c.customer_id = l.customer_id
)
SELECT
    `CIBIL Brackets`,
    ROUND(SUM(CASE WHEN loan_status = 'Defaulted' THEN loan_amount * 1.0 ELSE 0 END) /
    SUM(loan_amount) * LOG(COUNT(*)), 2) AS `Risk Efficiency`
FROM cibil_data
GROUP BY `CIBIL Brackets`
ORDER BY `Risk Efficiency` DESC;
```

STEP 3D — Income and Employment Risk Profile

```
-- Analyze loan portfolio and defaults by employment type
WITH employment_stats AS (
    SELECT
        c.employemnet_type,
        SUM(l.loan_amount) / 10000000.0 AS portfolio_amount_cr,
        SUM(CASE WHEN l.loan_status = 'Defaulted' THEN l.loan_amount * 1.0 ELSE 0 END) / 10000000.0 AS
defaulted_amount_cr,
        COUNT(*) AS total_loans,
        SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) AS default_count,
        ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS default_rate
    FROM customers c
    INNER JOIN loans l
        ON c.customer_id = l.customer_id
    GROUP BY c.employemnet_type
)
SELECT
    employement_type AS `Employment Type`,
    CONCAT(ROUND(portfolio_amount_cr, 2), ' Cr') AS `Portfolio Amount`,
    CONCAT(ROUND(defaulted_amount_cr, 2), ' Cr') AS `Defaulted Amount`,
    total_loans AS `Loans`,
    default_count AS `Default Count`,
    CONCAT(default_rate, '%') AS `Default Rate (%)`
FROM employment_stats
ORDER BY default_count DESC;

-- Analyze loan portfolio and defaults by income brackets
WITH income_brackets AS (
    SELECT
        customer_id,
```

```
        CASE
            WHEN CAST(annual_income AS DOUBLE) < 300000 THEN 'Low Income (<3L)'
            WHEN CAST(annual_income AS DOUBLE) >= 300000 AND CAST(annual_income AS DOUBLE) <= 800000 THEN 'Mid
(3L-7L)'
            ELSE 'High Income (>8L)'
        END AS income_bracket
    FROM customers
),
portfolio_stats AS (
    SELECT
        ib.income_bracket,
        SUM(l.loan_amount) / 10000000.0 AS portfolio_amount_cr,
        SUM(CASE WHEN l.loan_status = 'Defaulted' THEN l.loan_amount * 1.0 ELSE 0 END) / 10000000.0 AS
defaulted_amount_cr,
        COUNT(*) AS total_loans,
        SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) AS default_count,
        ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS default_rate
    FROM income_brackets ib
    INNER JOIN loans l
        ON ib.customer_id = l.customer_id
    GROUP BY ib.income_bracket
)
SELECT
    income_bracket AS `Income Bracket`,
    CONCAT(ROUND(portfolio_amount_cr, 2), ' Cr') AS `Portfolio Amount`,
    CONCAT(ROUND(defaulted_amount_cr, 2), ' Cr') AS `Defaulted Amount`,
    total_loans AS `Loans`,
    default_count AS `Default Count`,
    CONCAT(default_rate, '%') AS `Default Rate (%)`
FROM portfolio_stats
ORDER BY default_count DESC;
```

Employment Type	Portfolio Amount	Defaulted Amount	Loans	Default Count	Default Rate (%)
Private Employee	94.3 Cr	11.2 Cr	2291	275	12.0%
Self Employed	39.56 Cr	6.91 Cr	967	168	17.37%
Government Employee	39.61 Cr	2.75 Cr	958	62	6.47%
Business Owner	23.5 Cr	2.08 Cr	554	46	8.3%
Student	7.86 Cr	1.3 Cr	230	39	16.96%

STEP 3E — Prioritized Collections Target List (Final Output)

```
WITH customer_defaults AS (
    SELECT
        c.customer_id,
        c.full_name,
        c.phone_number,
        c.email_address,
        c.cibil_score,
        COUNT(l.loan_id) AS total_loans,
        CONCAT(ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN l.loan_amount * 1.0 ELSE 0 END) / 100000.0,
2), ' L') AS default_amount,
        CONCAT(ROUND(SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) * 100 / COUNT(*), 2), '%') AS
default_rate
    FROM customers c
    JOIN loans l
        ON c.customer_id = l.customer_id
    GROUP BY
        c.customer_id,
        c.full_name,
        c.phone_number,
        c.email_address,
        c.cibil_score
),
scored AS (
    SELECT
        *,
        (
            CAST(REPLACE(default_amount, ' L', '') AS DOUBLE) *
            CAST(REPLACE(default_rate, '%', '') AS DOUBLE)
        ) AS score
    FROM customer_defaults
)
SELECT
    customer_id AS `Customer ID`,
    full_name AS `Name`,
    phone_number AS `Contact`,
    email_address AS `Email`,
    default_amount AS `Defaulted Amount`,
    default_rate AS `Default Rate (%)`,
    cibil_score AS `CIBIL Score`,
    ROUND(score, 2) AS `Priority Score`
FROM scored
ORDER BY score DESC
LIMIT 50;
```

Customer ID	Name	Contact	Email	Defaulted Amount	Default Rate (%)	CIBIL Score	Priority Score
1047	Raghav Dubey	918494902096	raghav.dubey194@yahoo.com	15.07 L	100.0%	486	1507.0
4347	Max Pillay	919575772783	max.pillay122@hotmail.com	11.7 L	100.0%	730	1170.0
1666	Saumya Sarraf	918731863798	saumya.sarraf606@yahoo.com	11.51 L	100.0%	711	1151.0
2935	Mohammed Mammen	918820058128	mohammed.mammen602@outlook.com	11.4 L	100.0%	557	1140.0
3102	Balveer Swamy	918071485656	balveer.swamy341@gmail.com	11.06 L	100.0%	560	1106.0
2998	Peter Sarma	919799532101	peter.sarma835@hotmail.com	10.37 L	100.0%	669	1037.0
4620	Pushti Doctor	918092197558	pushti.doctor209@outlook.com	10.28 L	100.0%	637	1028.0
1128	Ronith Anne	918875996993	ronith.anne920@outlook.com	10.18 L	100.0%	582	1018.0
3030	Yashvi Kuruvilla	919798989829	yashvi.kuruvilla313@outlook.com	10.07 L	100.0%	714	1007.0
2463	Hamsini Wadhwa	917596992261	hamsini.wadhwa608@outlook.com	9.93 L	100.0%	541	993.0
992	Jeet Randhawa	919140933057	jeet.randhawa655@yahoo.com	9.85 L	100.0%	580	985.0
4724	Ishani Joshi	917231771111	ishani.joshi420@outlook.com	9.68 L	100.0%	583	968.0
3761	Pratyush Thakkar	919231949080	pratyush.thakkar69@gmail.com	9.55 L	100.0%	586	955.0
960	Faqid Doshi	919099636955	faqid.doshi222@gmail.com	9.4 L	100.0%	534	940.0
4154	Rachit Pingle	919695878341	rachit.pingle933@gmail.com	9.28 L	100.0%	716	928.0
3821	Dakshesh Purohit	919054366215	dakshesh.purohit663@hotmail.com	13.63 L	66.67%	685	908.71
1852	Henry Patel	917922107899	henry.patel183@outlook.com	8.81 L	100.0%	724	881.0
3283	Tripti Khatri	917332209853	tripti.khatri977@gmail.com	8.76 L	100.0%	749	876.0
4431	Amol Nayak	917476703358	amol.nayak830@yahoo.com	8.74 L	100.0%	630	874.0
3298	Yashasvi Sundaram	917611212814	yashasvi.sundaram746@outlook.com	12.97 L	66.67%	598	864.71

Custom Column

```
WITH defaulted_customer_data AS (
    SELECT
        c.customer_id,
        c.full_name,
        c.cibil_score,
        c.annual_income,
        SUM(l.loan_amount) AS total_loan_amount,
        SUM(CASE WHEN l.loan_status = 'Defaulted' THEN l.loan_amount * 1.0 ELSE 0 END) AS defaulted_amount,
        SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) AS default_count
    FROM customers c
    INNER JOIN loans l
        ON c.customer_id = l.customer_id
    GROUP BY c.customer_id, c.full_name, c.cibil_score, c.annual_income
    HAVING SUM(CASE WHEN l.loan_status = 'Defaulted' THEN 1 ELSE 0 END) > 0
),
risk_score_data AS (
    SELECT
        *,
        ROUND(
            0.5 * (defaulted_amount / 100000.0) +
```

```
            0.3 * default_count * 100000 +
            0.2 * (900 - cibil_score) * 1000    ,
        2) AS risk_score
    FROM defaulted_customer_data
),
risk_segments AS (
    SELECT
        *,
        AVG(risk_score) OVER () AS avg_risk,
        NTILE(3) OVER (ORDER BY risk_score DESC) AS risk_ntile,
        CASE
            WHEN NTILE(3) OVER (ORDER BY risk_score DESC) = 1 THEN 'High Risk'
            WHEN NTILE(3) OVER (ORDER BY risk_score DESC) = 2 THEN 'Medium Risk'
            ELSE 'Low Risk'
        END AS borrower_segment
    FROM risk_score_data
),
collection_strategy AS (
    SELECT
        *,
        CASE
            WHEN risk_score >= avg_risk THEN 'Above Average Risk'
            ELSE 'Below Average Risk'
        END AS relative_risk,
        CASE
            WHEN borrower_segment = 'High Risk' AND risk_score >= avg_risk
                THEN 'Immediate Legal Action'
            WHEN borrower_segment = 'High Risk'
                THEN 'Escalated Collection'
            WHEN borrower_segment = 'Medium Risk' AND risk_score >= avg_risk
                THEN 'Legal Notice'
            WHEN borrower_segment = 'Medium Risk'
                THEN 'Aggressive Collection'
            WHEN borrower_segment = 'Low Risk' AND risk_score >= avg_risk
                THEN 'Payment Plan Negotiation'
            ELSE 'Standard Recovery'
        END AS collection_approach
    FROM risk_segments
)
SELECT
    customer_id AS `Customer ID`,
    full_name AS `Customer Name`,
    CONCAT(ROUND(total_loan_amount / 100000.0, 2), ' L') AS `Loan Amount`,
    CONCAT(ROUND(defaulted_amount / 100000.0, 2), ' L') AS `Default Amount`,
    cibil_score AS `CIBIL`,
    CONCAT(ROUND(annual_income / 100000.0, 2), ' L') AS `Annual Income`,
    default_count AS `Default Count`,
    risk_score AS `Risk Score`,
    borrower_segment AS `Borrower Segment`,
    relative_risk AS `Relative Risk`,
    collection_approach AS `Collection Approach`
FROM collection_strategy
ORDER BY risk_score DESC;

CASE
```

```
  WHEN borrower_segment = 'High Risk' AND risk_score >= avg_risk
    THEN 'Immediate Legal Action'
  WHEN borrower_segment = 'High Risk'
    THEN 'Escalated Collection'
  WHEN borrower_segment = 'Medium Risk' AND risk_score >= avg_risk
    THEN 'Legal Notice'
  WHEN borrower_segment = 'Medium Risk'
    THEN 'Aggressive Collection'
  WHEN borrower_segment = 'Low Risk' AND risk_score >= avg_risk
    THEN 'Payment Plan Negotiation'
  ELSE 'Standard Recovery'
END AS collection_approach
```

> CTEs are better because they break complex logic into readable, named stages rather than burying it in nested CASE statements. This makes debugging easier (you can inspect each CTE layer) and lets the database reuse intermediate results instead of recalculating them.

WHY Gate

1. If you added employment_type as a third factor in borrower_segment — would any customers move to a different segment? Pick one specific customer and show the re-classification logic.

> In my current logic to calculating borrower segment, i have used the risk score (default_amount, default_count & annual income) and not CIBIL brackets. When employment_type is added, it creates a composite_risk_score through a weighted formula, which changes the customer ranking order, customers with better employment stability get lower composite scores and may shift from High Risk to Medium Risk if their new rank crosses the NTILE boundary.

2. Your top 50 list: the collections team can make 200 calls per week. How do you prioritise the order within your top 50? Is priority_score alone sufficient?

> I have ordered my top 50 list using a priority_score calculated by multiplying defaulted_amount and default_rate for each customer. This is sufficient because the collections team makes 200 calls/week, exactly the capacity for a top 50 list, and the priority_score directly optimizes for recovery: customers with large outstanding defaults AND high default rates rank first,

> combining financial exposure with behavioral risk. The team works top-to-bottom knowing each customer represents maximum recovery potential, eliminating the need for additional manual prioritization or tie-breakers.

3. Write the "Riskiest Borrower Profile" paragraph. Describe the type of customer who appears most frequently in your top segment.

> The riskiest borrower profile is typically a self-employed or student customer with a low to mid CIBIL score and multiple past defaults. These borrowers often have higher defaulted loan amounts and repeated default behavior, indicating weak repayment stability. They appear most frequently in the top risk segment due to both financial exposure and unstable income sources.

© 2025 Skill AI Path